

EN3160 Assignment 1 on Intensity Transformations and Neighborhood Filters

Index No: 230563H

** Q1.**

```
In [265... import matplotlib.pyplot as plt; plt.rcParams.update({'font.size': 6, 'axes.titlesi
import cv2
import numpy as np
def intensity_transformation(im,breakpoints):
    x,y=zip(*breakpoints)
    t_im=np.interp(im,x,y)
    return t_im
breakpoints_def = [(0,0), (50,50), (100,150), (150,255), (255,255)]
im=cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensity tran
t_im_def=intensity_transformation(im,breakpoints_def)
plt.figure(figsize=(6.5, 1.7)); plt.subplot(1,2,1); plt.imshow(im,cmap='gray',vmin=
```

Original



Default (Fig 1a)



```
In [266... # Part 2: Custom Breakpoints Experiment
breakpoints1 = [(0, 0), (60, 95), (170, 185), (255, 255)]
breakpoints2 = [(0, 0), (80, 130), (190, 205), (255, 255)]
breakpoints3 = [(0, 15), (50, 80), (180, 195), (255, 255)]
breakpoints4 = [(0, 0), (30, 80), (100, 100), (255, 255)]
t_im1 = intensity_transformation(im, breakpoints1)
t_im2 = intensity_transformation(im, breakpoints2)
t_im3 = intensity_transformation(im, breakpoints3)
t_im4 = intensity_transformation(im, breakpoints4)
plt.figure(figsize=(6.5, 1.7)); plt.subplot(1,4,1); plt.imshow(t_im1,cmap='gray',vm
```

BP1



BP2



BP3



BP4 (Best)

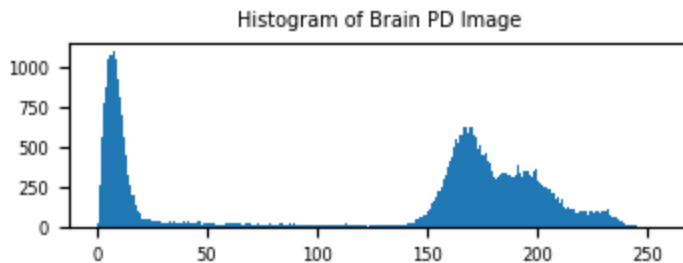


Experiment: Breakpoints 1-3 provide various contrasts. The edited `breakpoints4` yields the clearest image by aggressively lifting shadows to make the full face observable, while preserving lighter areas without clipping.

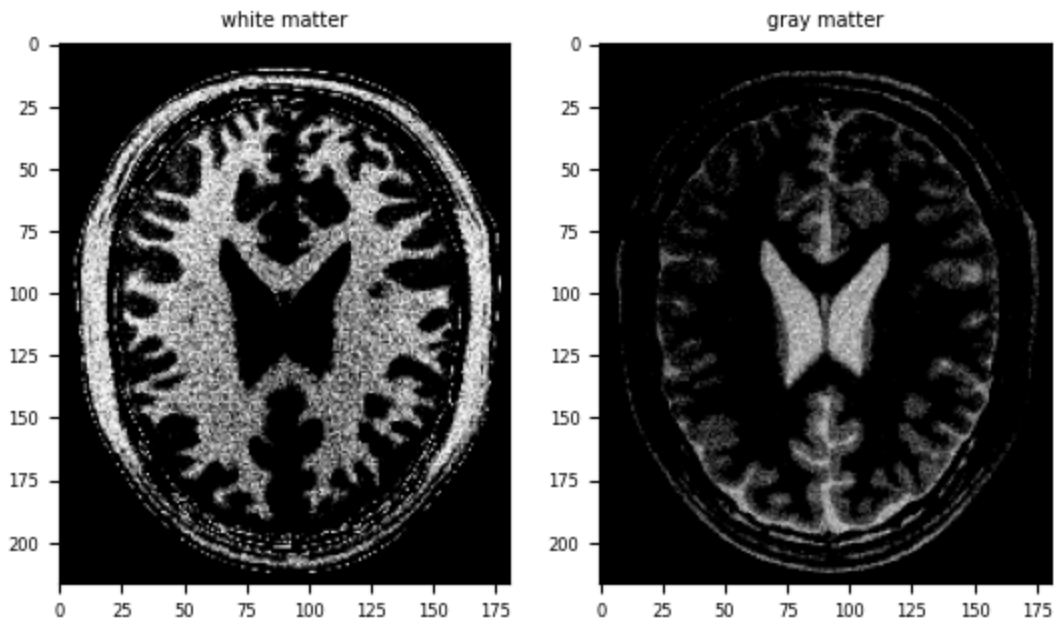
**** Q2.****

```
In [267... from skimage.filters import threshold_multiotsu
brain_img = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\inten
thresholds = threshold_multiotsu(brain_img, classes=4)
t1, t2, t3 = thresholds
print(f"Thresholds: {t1}, {t2}, {t3}")
plt.figure(figsize=(4, 1.2)); plt.hist(brain_img.ravel(), bins=256, range=[0,256]);
```

Thresholds: 45, 124, 186



```
In [268... breakpointwhite=[(0, 0), (124, 0), (155, 255), (186, 0), (255, 0)]
breakpointgray=[(0, 0), (186, 0), (255, 255)]
t_im_white=intensity_transformation(brain_img,breakpointwhite)
t_im_gray=intensity_transformation(brain_img,breakpointgray)
plt.subplot(1,2,1); plt.imshow(t_im_white,cmap='gray',vmin=0,vmax=255); plt.title("white matter")
plt.subplot(1,2,2); plt.imshow(t_im_gray,cmap='gray',vmin=0,vmax=255); plt.title("gray matter")
```



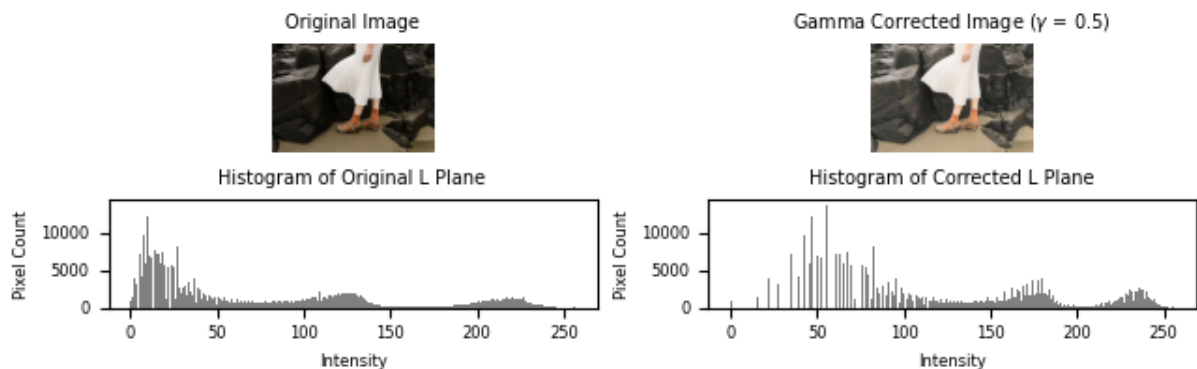
In this MRI modality, pixel intensity correlates directly with tissue water content. Multi-Otsu thresholding effectively segments the anatomy because the tissues cluster into distinct histogram peaks: lipid-rich white matter possesses lower water content and occupies the darker intensity band, while water-rich gray matter occupies the brighter mid-intensity band.

Cerebrospinal fluid (CSF) forms the brightest peak. The piecewise linear transformations are then utilized to isolate and expand the contrast of these specific biological intensity brackets.

**** Q3.****

In [269...

```
img_bgr = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensi
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(img_lab)
gamma = 0.5
L_normalized = L / 255.0
L_corrected = (L_normalized ** gamma) * 255.0
L_corrected = np.clip(L_corrected, 0, 255).astype(np.uint8)
img_lab_corrected = cv2.merge((L_corrected, a, b))
img_rgb_original = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_rgb_corrected = cv2.cvtColor(img_lab_corrected, cv2.COLOR_LAB2RGB)
plt.figure(figsize=(6.5, 2.1)); plt.subplot(2, 2, 1); plt.imshow(img_rgb_original);
```



A gamma value of $\gamma = 0.5$ was selected for the L plane correction. Because this value is less than 1, it applies a non-linear transformation that boosts lower-intensity pixels (recovering details in the dark foreground rocks) while leaving higher-intensity pixels (the bright skirt) relatively unchanged.

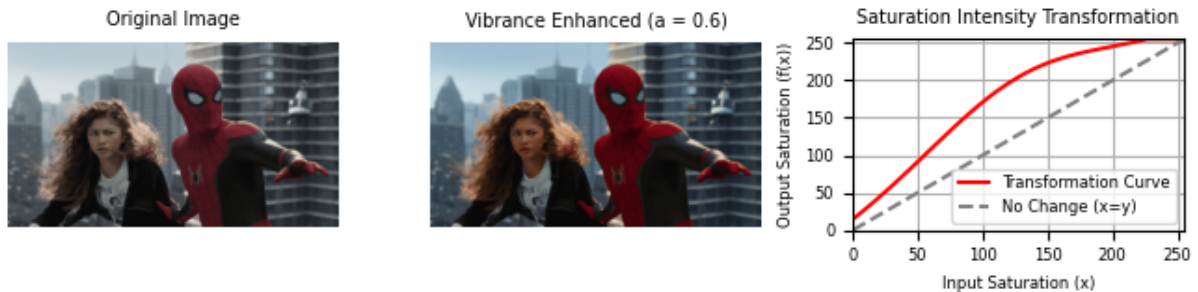
Operating strictly in the $L^*a^*b^*$ lightness channel ensures that these shadows are brightened without distorting the original hue and saturation. Consequently, the histograms reflect a clear shift toward higher lightness values.

**** Q4.****

In [270...

```
img_bgr = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensi
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(img_hsv)
a = 0.6
sigma = 70
s_float = s.astype(np.float32)
boost = a * 128.0 * np.exp(-((s_float - 128.0)**2) / (2 * sigma**2))
s_enhanced = s_float + boost
s_enhanced = np.clip(s_enhanced, 0, 255).astype(np.uint8)
img_hsv_enhanced = cv2.merge((h, s_enhanced, v))
img_rgb_original = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_rgb_enhanced = cv2.cvtColor(img_hsv_enhanced, cv2.COLOR_HSV2RGB)
```

```
plt.figure(figsize=(6.5, 1.7)); plt.subplot(1, 3, 1); plt.imshow(img_rgb_original);
x_vals = np.arange(0, 256, dtype=np.float32)
y_vals = x_vals + a * 128.0 * np.exp(-((x_vals - 128.0)**2) / (2 * sigma**2))
y_vals = np.clip(y_vals, 0, 255)
plt.subplot(1, 3, 3); plt.plot(x_vals, y_vals, color='red', label='Transformation C
```

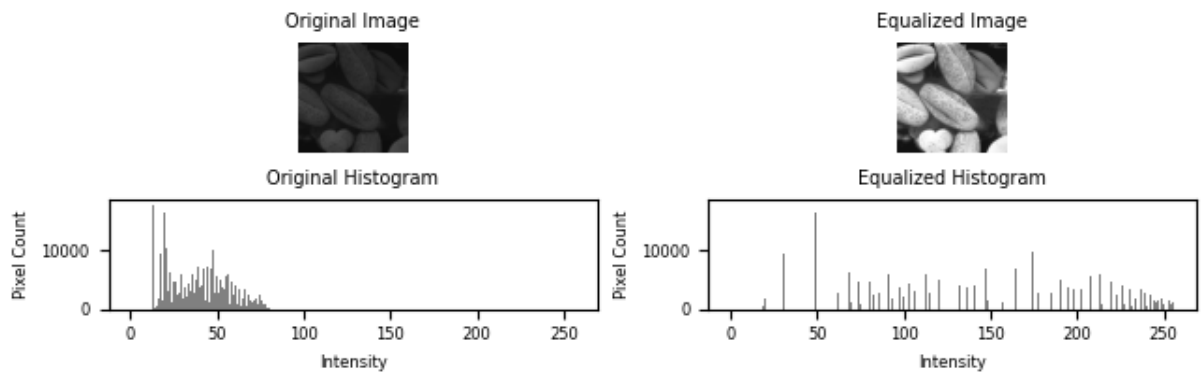


Unlike standard saturation adjustments that can cause oversaturation, this Gaussian-weighted vibrance function selectively boosts the saturation of less-saturated pixels (centered around mid-saturation 128). This effectively enhances the image's overall vibrancy while leaving already highly saturated pixels intact, resulting in natural-looking skin tones.

**** Q5.****

In [271...

```
def custom_histogram_equalization(img):
    histo = {i: 0 for i in range(256)}
    rows, cols = img.shape
    total_pixels = rows * cols
    for r in range(rows):
        for c in range(cols):
            pixel_val = img[r, c]
            histo[pixel_val] += 1
    new_pixel_values = {}
    cumulative_prob = 0.0
    for i in range(256):
        prob = histo[i] / total_pixels
        cumulative_prob += prob
        new_pixel_values[i] = round(cumulative_prob * 255)
    img_eq = np.zeros_like(img)
    for r in range(rows):
        for c in range(cols):
            old_val = img[r, c]
            img_eq[r, c] = new_pixel_values[old_val]
    return img_eq.astype(np.uint8)
img_gray = cv2.imread('C:\\Users\\Jaindu Samaranayaka\\Documents\\Semester_5\\CV\\i
img_eq = custom_histogram_equalization(img_gray)
plt.figure(figsize=(6.5, 2.1)); plt.subplot(2, 2, 1); plt.imshow(img_gray, cmap='gr
```



The original image suffers from low contrast, as evidenced by its concentrated histogram. Histogram equalization computes a transformation based on the cumulative distribution function (CDF) to spread the most frequent intensity values across the entire 0-255 range. This dramatically improves global contrast and reveals finer underlying textures.

**** Q6.****

```
In [272... img_bgr = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensi
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(img_hsv)
_, mask_initial = cv2.threshold(s, 15, 255, cv2.THRESH_BINARY)
kernel = np.ones((7, 7), np.uint8)
mask = cv2.morphologyEx(mask_initial, cv2.MORPH_CLOSE, kernel, iterations=2)
fg_v = cv2.bitwise_and(v, v, mask=mask)
hist, bins = np.histogram(fg_v[mask > 0], bins=256, range=[0, 256])
cdf = np.cumsum(hist)
MN = np.sum(hist)
cdf_normalized = (255 / MN) * cdf
cdf_final = np.round(cdf_normalized).astype(np.uint8)
eq_fg_v = np.zeros_like(v)
eq_fg_v[mask > 0] = cdf_final[v[mask > 0]]
bg_mask = cv2.bitwise_not(mask)
bg_v = cv2.bitwise_and(v, v, mask=bg_mask)
v_final = cv2.add(eq_fg_v, bg_v)
hsv_final = cv2.merge((h, s, v_final))
img_result_rgb = cv2.cvtColor(hsv_final, cv2.COLOR_HSV2RGB)
plt.figure(figsize=(6.5, 1.7)); plt.subplot(2, 3, 1); plt.imshow(h, cmap='gray', vm
```

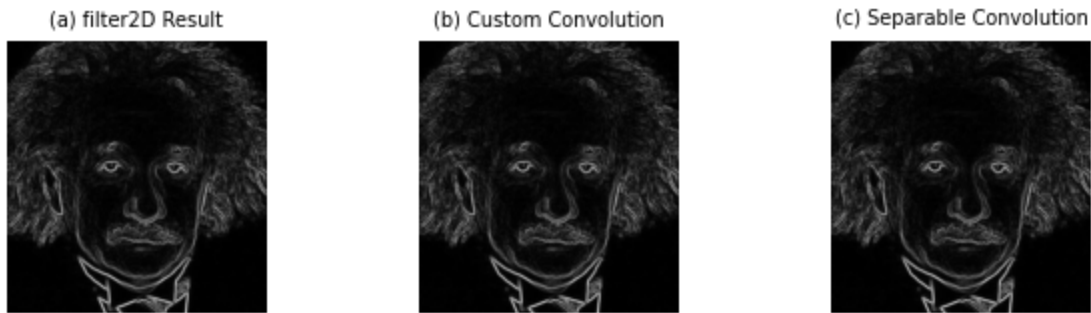


Global histogram equalization can sometimes wash out backgrounds. By extracting a binary mask, we isolate the foreground subject. Morphological closing (cv2.MORPH_CLOSE) is a necessary step to fill low-saturation 'holes' in the mask (such as eyes and teeth) before proceeding. Equalizing only this solid foreground region and recombining it with the original background yields highly pronounced subject features without distorting the surrounding context.

**** Q7.****

In [273...

```
img = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensity t
sobel_x = np.array([[1, 0, -1],
                    [2, 0, -2],
                    [1, 0, -1]], dtype=np.float32)
sobel_y = np.array([[1, 2, 1],
                    [0, 0, 0],
                    [-1, -2, -1]], dtype=np.float32)
gx_a = cv2.filter2D(img, -1, sobel_x)
gy_a = cv2.filter2D(img, -1, sobel_y)
mag_a = np.sqrt(gx_a**2 + gy_a**2)
def custom_convolve(image, kernel):
    k_h, k_w = kernel.shape
    pad_h, pad_w = k_h // 2, k_w // 2
    padded_img = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='edge')
    output = np.zeros_like(image)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            roi = padded_img[i:i+k_h, j:j+k_w]
            output[i, j] = np.sum(roi * kernel)
    return output
gx_b = custom_convolve(img, sobel_x)
gy_b = custom_convolve(img, sobel_y)
mag_b = np.sqrt(gx_b**2 + gy_b**2)
col_x = np.array([[1], [2], [1]], dtype=np.float32)
row_x = np.array([[1, 0, -1]], dtype=np.float32)
col_y = np.array([[1], [0], [-1]], dtype=np.float32)
row_y = np.array([[1, 2, 1]], dtype=np.float32)
temp_gx = custom_convolve(img, col_x)
gx_c = custom_convolve(temp_gx, row_x)
temp_gy = custom_convolve(img, col_y)
gy_c = custom_convolve(temp_gy, row_y)
mag_c = np.sqrt(gx_c**2 + gy_c**2)
def normalize(mag):
    mag_norm = (mag / mag.max()) * 255
    return np.clip(mag_norm, 0, 255).astype(np.uint8)
plt.figure(figsize=(6.5, 1.7)); plt.subplot(1, 3, 1); plt.imshow(normalize(mag_a),
```



The Sobel operator approximates the image gradient to effectively detect edges. Due to the mathematical property of separability, the 2D Sobel kernel can be decomposed into a 1D column and a 1D row vector. This significantly reduces computational complexity from $O(K^2)$ to $O(2K)$ per pixel. The resulting gradient magnitude clearly highlights structural edges.

**** Q8.****

In [274...

```
import os
base_dir = r"C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensity transfo
img_large_path = os.path.join(base_dir, "fig7.png")
img_small_path = os.path.join(base_dir, "fig7_small.png")
img_large = cv2.imread(img_large_path)
if img_large is None:
    print(f"Error: Could not load image at {img_large_path}. Please check the filen
else:
    h, w = img_large.shape[:2]
    img_small = cv2.resize(img_large, (w // 4, h // 4), interpolation=cv2.INTER_ARE
    cv2.imwrite(img_small_path, img_small)
    def custom_zoom(img, s, method='nearest'):
        h, w = img.shape[:2]
        new_h, new_w = int(h * s), int(w * s)
        x_new, y_new = np.meshgrid(np.arange(new_w), np.arange(new_h))
        x_old = x_new / s
        y_old = y_new / s
        if method == 'nearest':
            x_nearest = np.clip(np.round(x_old).astype(int), 0, w - 1)
            y_nearest = np.clip(np.round(y_old).astype(int), 0, h - 1)
            if len(img.shape) == 3:
                return img[y_nearest, x_nearest, :]
            else:
                return img[y_nearest, x_nearest]
        elif method == 'bilinear':
            x_floor = np.clip(np.floor(x_old).astype(int), 0, w - 1)
            x_ceil = np.clip(np.ceil(x_old).astype(int), 0, w - 1)
            y_floor = np.clip(np.floor(y_old).astype(int), 0, h - 1)
            y_ceil = np.clip(np.ceil(y_old).astype(int), 0, h - 1)
            dx = x_old - x_floor
            dy = y_old - y_floor
            if len(img.shape) == 3:
                dx = dx[..., np.newaxis]
                dy = dy[..., np.newaxis]
            top_left = img[y_floor, x_floor].astype(np.float32)
```

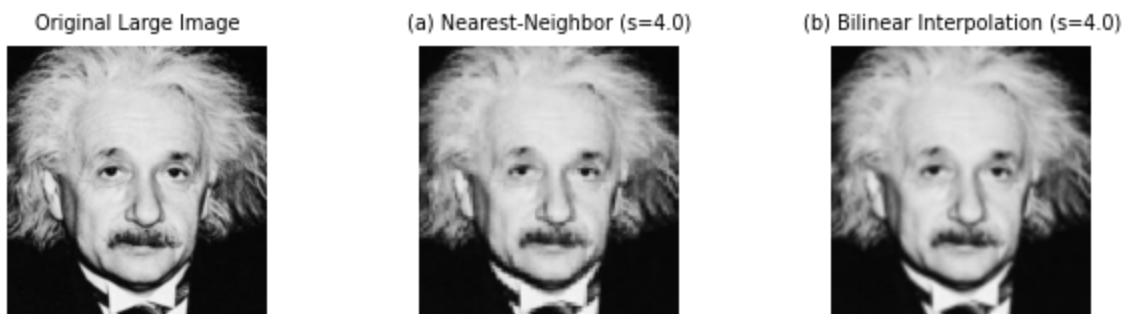
```

        top_right = img[y_floor, x_ceil].astype(np.float32)
        bottom_left = img[y_ceil, x_floor].astype(np.float32)
        bottom_right = img[y_ceil, x_ceil].astype(np.float32)
        r1 = top_left * (1 - dx) + top_right * dx
        r2 = bottom_left * (1 - dx) + bottom_right * dx
        interpolated = r1 * (1 - dy) + r2 * dy
        return np.clip(interpolated, 0, 255).astype(np.uint8)
def calculate_nssd(original, zoomed):
    orig_f = original.astype(np.float64)
    zoom_f = zoomed.astype(np.float64)
    min_h = min(orig_f.shape[0], zoom_f.shape[0])
    min_w = min(orig_f.shape[1], zoom_f.shape[1])
    orig_f = orig_f[:min_h, :min_w]
    zoom_f = zoom_f[:min_h, :min_w]
    ssd = np.sum((orig_f - zoom_f) ** 2)
    norm_factor = np.sum(orig_f ** 2)
    return ssd / norm_factor
s = 4.0
zoomed_nn = custom_zoom(img_small, s, method='nearest')
zoomed_bl = custom_zoom(img_small, s, method='bilinear')
nssd_nn = calculate_nssd(img_large, zoomed_nn)
nssd_bl = calculate_nssd(img_large, zoomed_bl)
print(f"Normalized SSD (Nearest-Neighbor): {nssd_nn:.6f}")
print(f"Normalized SSD (Bilinear): {nssd_bl:.6f}")
plt.figure(figsize=(6.5, 1.7)); plt.subplot(1, 3, 1); plt.imshow(cv2.cvtColor(img_1

```

Normalized SSD (Nearest-Neighbor): 0.019940

Normalized SSD (Bilinear): 0.014966

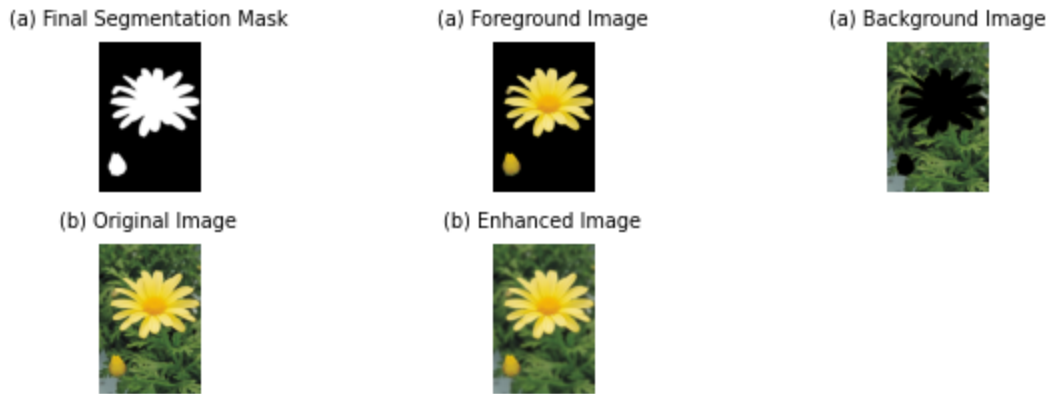


Nearest-neighbor interpolation simply duplicates the closest original pixel, resulting in a fast upscaled image that suffers from aliasing (pixelation artifacts). Bilinear interpolation computes a weighted average of the four nearest pixels, acting as a mild low-pass filter that smooths the edges but slightly reduces overall high-frequency sharpness. The Normalized Sum of Squared Differences (SSD) confirms that the bilinear approach yields a lower error and higher fidelity to the original image.

**** Q 9.****

In [275... `img_bgr = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensi`
`img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)`
`h, w = img_rgb.shape[:2]`
`rect = (10, 10, w - 20, h - 20)`
`mask = np.zeros(img_rgb.shape[:2], np.uint8)`
`bgdModel = np.zeros((1, 65), np.float64)`

```
fgdModel = np.zeros((1, 65), np.float64)
cv2.grabCut(img_rgb, mask, rect, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_RECT)
binary_mask = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
foreground = img_rgb * binary_mask[:, :, np.newaxis]
background = img_rgb * (1 - binary_mask[:, :, np.newaxis])
blurred_base = cv2.GaussianBlur(img_rgb, (35, 35), 0)
blurred_background = blurred_base * (1 - binary_mask[:, :, np.newaxis])
enhanced_img = foreground + blurred_background
plt.figure(figsize=(6.5, 2.1)); plt.subplot(2, 3, 1); plt.imshow(binary_mask, cmap=
```



GrabCut successfully segments the sharp foreground flower. By heavily blurring the background and recombining it, we simulate a shallow depth-of-field effect. **(c)** The dark border around the flower is a phenomenon known as the halo effect or edge leakage. It occurs because the blurring operation on the background bleeds the dark boundary pixels of the foreground into the surrounding background.

**** Q10.****

In [276...

```
img_bgr = cv2.imread(r'C:\Users\Jaindu Samaranayaka\Documents\Semester_5\CV\intensi
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
d = 9
sigma_s = 75.0
sigma_r = 75.0
blur_gaussian = cv2.GaussianBlur(img_rgb, (d, d), sigma_s)
bilateral_cv = cv2.bilateralFilter(img_rgb, d, sigma_r, sigma_s)
def custom_bilateral(img, d, sigma_r, sigma_s):
    img_f = img.astype(np.float32)
    h, w, c = img.shape
    pad = d // 2
    padded = np.pad(img_f, ((pad, pad), (pad, pad), (0, 0)), mode='reflect')
    output = np.zeros_like(img_f)
    y, x = np.ogrid[-pad:pad+1, -pad:pad+1]
    spatial_w = np.exp(-(x**2 + y**2) / (2 * sigma_s**2))
    for i in range(h):
        for j in range(w):
            patch = padded[i:i+d, j:j+d, :]
            center_pixel = padded[i+pad, j+pad, :]
            color_diff = patch - center_pixel
            color_dist_sq = np.sum(color_diff**2, axis=-1)
            range_w = np.exp(-color_dist_sq / (2 * sigma_r**2))
            w_total = spatial_w * range_w
```

```

        for k in range(c):
            output[i, j, k] = np.sum(w_total * patch[:, :, k]) / np.sum(w_total)
        return np.clip(output, 0, 255).astype(np.uint8)
bilateral_custom = custom_bilateral(img_rgb, d, sigma_r, sigma_s)
def calculate_mse(img1, img2):
    return np.mean((img1.astype(np.float64) - img2.astype(np.float64))**2)
mse_value = calculate_mse(bilateral_cv, bilateral_custom)
print(f"Mean Squared Error (MSE) between OpenCV and Custom Bilateral: {mse_value:.4}")
plt.figure(figsize=(6.5, 2.1)); plt.subplot(2, 3, 1); plt.imshow(img_rgb); plt.titl

```

Mean Squared Error (MSE) between OpenCV and Custom Bilateral: 28.7494



While Gaussian blurring smooths the image uniformly and degrades important structural edges, bilateral filtering overcomes this by weighting pixels based on both spatial proximity and intensity similarity. This allows the bilateral filter to heavily smooth homogenous regions (reducing noise) while preserving sharp, abrupt intensity changes.

GitHub Repository: [EN3160-A01-Intensity-Transformations-and-Neighborhood-Filtering](#)